Extracting decimal digits of π in quasi-linear time from binary BBP formulas

Joseph Babčanec
Department of Mathematics, Benedict College
Columbia, SC, USA
joseph.babcanec@benedict.edu

July 2026

Abstract

The Bailey–Borwein–Plouffe (BBP) formula computes an isolated hexadecimal digit of π without the earlier digits, but no decimal analogue exists: Borwein, Borwein and Galway proved that π has no Machin-type BBP formula in a base that is not a power of 2, and every published decimal algorithm runs in essentially quadratic time. We compute the N-th decimal digit of π from the ordinary hexadecimal BBP formula in quasi-linear time, in $O(N \log^3 N)$ bit operations and $\Theta(N)$ bits of space. That space is essentially the binary expansion of one integer, $5^{\,N-1}$, shared by all terms: after the series is multiplied by 10^{N-1}, an exact identity recovers each term's fractional part from residues and bit-windows of the shared integer. The method works for any base-2^r BBP constant, such as $\ln 2$, gives digits in any base, and repairs the base-5 obstruction of Zudilin (arXiv:2409.10097). Python and C++ implementations are verified against public digit corpora through position 10^7, at a measured exponent of $N^{1.077}$ against $N^{1.8}$ for the strongest published method; to our knowledge this is the first implemented and verified subquadratic decimal extractor. It sits one logarithmic factor above the $O(N \log^2 N)$ cost of computing all N digits by the arithmetic–geometric mean. We isolate that factor as a single open problem, the computation of a consolidated 2-adic integer V: its multiplicative part falls to $O(N \log^2 N)$ by a classical factorization technique, while the additive part, equivalent to an isolated harmonic number modulo 2^N, has no known fast algorithm.

1 Introduction

Write $d_N(\xi)$ for the N-th decimal digit of $\xi > 0$ after the decimal point, so that with $\{x\} = x - \lfloor x \rfloor$,

$$d_N(\xi) \;=\; \big\lfloor 10\,\{10^{\,N-1}\xi\}\big\rfloor. \tag{1}$$

The *digit extraction problem* asks for $d_N(\pi)$ without computing the $N-1$ digits before it. Bailey, Borwein and Plouffe [1] solved the binary case with the formula

$$\pi \;=\; \sum_{k=0}^{\infty} \frac{1}{16^k}\Big(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6}\Big), \tag{2}$$

which yields hexadecimal digits at position N in $O(N \log N)$-type time and $O(\log N)$ space via per-term modular exponentiation. For base 10 no such formula exists: Borwein, Borwein and

Galway [2] proved that π has no Machin-type BBP arctangent formula in any base $b \neq 2^m$. The known decimal algorithms form a ladder of essentially quadratic (or worse) methods: Plouffe's original scheme [3] runs in $O(N^3 \log^3 N)$ time, Bellard's improvement [4] in $O(N^2)$ time in any base, and Gourdon's method [5] in $O\big(N^2 \log\log N / \log^2 N\big)$ word operations with $O(\log^2 N)$ memory. Plouffe's 2022 approach [6] extracts a digit from an asymptotic expansion whose coefficients are Bernoulli (or Euler) numbers; it requires precomputed tables of those numbers and is thereby capped near position 10^8, the current extent of Bernoulli computations. Most recently Zudilin [7] attempted a BBP-style computation of π in base 5 (the first arXiv version was titled "…in base 10") via a Salikhov-type representation over $\mathbb{Z}[i]$, and identified an "obvious flaw": his per-term modular identity fails exactly on the terms in which the power of 5 in the numerator is exhausted, and no truncation avoids them. He left the repair as an open question; the note has no published successor.

This paper repairs that flaw, not for Zudilin's Gaussian series specifically, but for the ordinary formula (2) and, with it, every binary BBP constant. The price is memory: we abandon the $O(\log N)$-space regime of BBP and store the binary expansion of the single integer $X = 5^{N-1}$, about $0.2902\,N$ bytes. In exchange the running time becomes quasi-linear.

Theorem 1 (main)**.** *Let $\mathsf{M}(n)$ denote the cost of multiplying two n-bit integers. The N-th decimal digit of π (together with any constant window of following digits) can be computed, without computing any earlier digit, in*

$$O\big(\mathsf{M}(N) \;+\; \mathsf{M}(N \log N) \log N \;+\; N \log^2 N \log\log N\big) \;\subseteq\; O\big(N \log^3 N\big)$$

bit operations, using $\Theta(N)$ bits of space: the read-only binary expansion of 5^{N-1}, shared by all terms, together with $O(N)$ bits of working space (Proposition 8). A space-minimal variant computes the same digit using only $O(\log^2 N)$ bits of writable space beyond the shared string, at the cost of quadratic time. The output is certified in the Las Vegas sense of Remark 6.

The same statement holds with π replaced by any constant possessing a BBP-type formula in a base 2^r (Theorem 15), for instance $\ln 2$; and with "decimal" replaced by "base B" for arbitrary $B \geq 2$ (Corollary 16), which supersedes the $O(N^2)$ any-base algorithm of [4] in time at the stated memory cost. Section 4 then sharpens the algorithm structurally: all phases except one collapse to $O(N \log\log N)$ word operations, and the surviving phase, the construction of a single 2-adic integer, is analyzed as a barrier in Section 6.

What is, and is not, claimed

By the Gauss–Legendre/AGM iteration of Brent and Salamin [10, 11], computing *all* N digits of π already takes $O(\mathsf{M}(N) \log N) = O(N \log^2 N)$ bit operations, and softly linear time by binary splitting of the Chudnovsky series, both in $\Theta(N)$ bits of memory; quasi-linear cost per isolated digit is therefore not by itself a new capability, and the natural benchmark for any extraction algorithm is *parity with full computation*: matching the $O(N \log^2 N)$ of the AGM while retaining the extraction semantics (no earlier digits produced, a read-only shared string, word-level parallelism). Our Theorem 1 sits one logarithmic factor above that benchmark. Separately, the canonical open problem posed in [1], decimal digit extraction in polylogarithmic (or even sublinear) space, remains open and is *not* resolved here, since our space is $\Theta(N)$ bits. The contributions are: (i) the first digit-extraction algorithm for decimal π based on a BBP formula, with per-term independence: after the one shared precomputation the terms are processed in

any order by any number of workers that communicate only word-size results, in the style of distributed binary-BBP projects; (ii) to our knowledge the first *implemented and verified* decimal extractor of complexity $N^{1+o(1)}$ (softly linear) in any model (Section 7); (iii) the resolution of the specific obstruction of [7]; (iv) an auxiliary object (one power of 5 computed on the fly) rather than tables that must be precomputed by other means, in contrast to [6]; (v) the sieve-batched exponentiation of Section 4, which also accelerates classical binary BBP extraction; and (vi) a precise, machine-validated characterization of the obstruction to going faster (Section 6). Closest to (ii) at the level of results, Gourdon's unpublished 2003 manuscript [5] asserts in its final section a time–memory tradeoff, $O\big(n^2 \log^3 n \log\log n/(m \log^2(n/m))\big)$ time at memory $m \in [n^{\varepsilon}, n]$, whose endpoint $m \approx n$ would likewise be softly linear; the manuscript states that "the details of the algorithm will be added soon in a next version," but no later version ever appeared (the archived file is unchanged from February 2003 through 2026), no details or implementation are known, and the sketch that is given (binary splitting over blocks) differs structurally from the present method. We therefore claim the algorithm and its analysis, not priority over the bare assertion that some such tradeoff should exist.

2 The three-phase decomposition

Throughout, $N \geq 1$ is the target position, $n := N - 1$, $X := 5^n$, and $L := \lceil n \log_2 5 \rceil$ the bit length of X. Since $x \mapsto \{x\}$ is a homomorphism $\mathbb{R} \to \mathbb{R}/\mathbb{Z}$, fractional parts of individual terms may be accumulated with signs modulo 1; alternating or negative coefficients pose no obstacle.

2.1 Normal form

Multiply (2) termwise by $10^n = 2^n 5^n$ and absorb all powers of two (the numerators $4, 2, 1, 1$ and the even parts of $8k+4 = 4(2k+1)$ and $8k+6 = 2(4k+3)$) into a single exponent. With $A := n - 4k$:

Lemma 2 (normal form)**.**

$$10^n \pi \;=\; \sum_{k \geq 0} \Big[\, 2^{A+2} \frac{X}{8k+1} \;-\; 2^{A-1} \frac{X}{2k+1} \;-\; 2^{A} \frac{X}{8k+5} \;-\; 2^{A-1} \frac{X}{4k+3} \Big], \tag{3}$$

i.e. every term equals $\pm 2^E X/\mu$ *with* μ *odd and* $E \in \{A+2,\, A-1,\, A,\, A-1\}$.

Proof. Immediate from $10^n 16^{-k} = 2^{n-4k} 5^n$ and the factorizations $8k+4 = 2^2(2k+1)$, $8k+6 = 2(4k+3)$. □

Terms decay like 16^{-k}, so positions N through $N + O(1)$ are determined by the terms with $k \leq K$, where $K = \lceil (n \log_2 10 + G + c)/4 \rceil$ for a G-bit accumulator (Section 2.5); in total $4(K+1) \approx 3.33\, N$ terms. Each term falls into one of three regimes.

2.2 Phase 1: easy terms ($E \geq 0$)

Here $2^E X$ is a nonnegative integer, so

$$\Big\{ \frac{2^E X}{\mu} \Big\} = \frac{\big(2^E \bmod \mu\big)\big(5^n \bmod \mu\big) \bmod \mu}{\mu},$$

computable with two modular exponentiations modulo the word-size integer μ, exactly as in classical BBP. There are $\Theta(N)$ such terms ($E \geq 0$ holds for $k \lesssim n/4$ in each family).

2.3 Phase 2: band terms ($E < 0$, $J := -E < L$)

These are the terms on which every previous decimal approach loses: the numerator is no longer an integer, the natural modulus $\mu 2^J$ grows exponentially along the family, and per-term reduction of X modulo $\mu 2^J$ costs $\Theta(\mathsf{M}(N))$, giving $\Theta(N)$-many big operations. This is the quadratic wall, and precisely the failure identified in [7, §3]. The wall disappears because *all band terms share the same numerator* X:

Lemma 3 (band identity)**.** *Let $X, \mu \geq 1$ and $J \geq 1$ be integers, $H := \lfloor X/2^J \rfloor$, $h := H \bmod \mu$, and $u := \{X/2^J\} \in [0, 1)$. Then*

$$\left\{ \frac{X}{\mu\, 2^J} \right\} \;=\; \frac{h+u}{\mu}.$$

Proof. $X/(\mu 2^J) = (H+u)/\mu$. Writing $H = q\mu + h$, $X/(\mu 2^J) = q + (h+u)/\mu$ with $0 \leq (h+u)/\mu < (\mu - 1 + 1)/\mu = 1$, so $(h+u)/\mu$ is the fractional part. □

Lemma 4 (recovering h)**.** *Let μ be odd, $R := X \bmod \mu$ and $\widetilde{Y} := \big(X \bmod 2^J\big) \bmod \mu$. Then*

$$h \;=\; \big(R - \widetilde{Y}\big) \cdot 2^{-J} \;\bmod\; \mu.$$

Proof. From $X = 2^J H + (X \bmod 2^J)$, reduce modulo μ and use that 2 is invertible mod μ. □

Consequently a band term needs: $R = 5^n \bmod \mu$ and $2^{-J} \bmod \mu = ((\mu+1)/2)^J \bmod \mu$, two word-size modular exponentiations; the *prefix residue* $\widetilde{Y}$, discussed in Section 3.2; and u to G_p bits, which is by definition the bit-window of X at depth J:

$$\big\lfloor 2^{G_p} u \big\rfloor = \left\lfloor \frac{X \bmod 2^J}{2^{\,J-G_p}} \right\rfloor = \text{bits } [\, J - G_p,\ J\,) \text{ of } X,$$

an $O(1)$ read from the stored binary expansion of X. This is the entire role of the $\Theta(N)$-bit auxiliary string: it converts every band term's "impossible" modulus $\mu 2^J$ into one shared object read locally.

2.4 Phase 3: tail terms ($J \geq L$)

Then $H = 0$, so the term is u/μ with $u = X/2^J < 1$ read from the top of the stored string; only $O(G)$ terms lie between $J = L$ and the accumulator cutoff.

2.5 Accumulation and error

All contributions are accumulated in a fixed-point register of G fraction bits, each term entering as $\lfloor 2^G (h+u)/\mu \rfloor$ (band/tail) or $\lfloor 2^G r/\mu \rfloor$ (easy), with sign, modulo 2^G.

Proposition 5 (error bound)**.** *With window precision G_p and T terms processed, the accumulator differs from $2^G\{10^n\pi\}$ by at most $T\big(1 + 2^{\,G-G_p}\big) + 2$ units, i.e. by $T \cdot 2^{-G} + T \cdot 2^{-G_p} + O(2^{-G})$ after rescaling. Choosing $G_p, G = \Theta(\log N)$ with $G_p \geq \log_2 T + g$ and $G \geq G_p + 48$ leaves g trustworthy guard bits above the output window.*

Proof. Each floor loses less than one unit in the last place; truncating u to G_p bits perturbs $(h+u)/\mu$ by less than $2^{-G_p}/\mu \leq 2^{-G_p}$; the series cutoff contributes less than one unit by the choice of K. Sum over T terms. □

Remark 6 (certification). As with all BBP-type extraction, the digit window is read off the top of the accumulator and is provably correct unless $\{10^n\pi\}$ lies within the total error ε of a carry boundary of the window, i.e. unless the digits following the window form a run of 9s or 0s of length $\approx (\log_{10}\varepsilon^{-1}) - w$. The algorithm detects this condition in the output and reruns with doubled G; under the standard heuristic that the digits of π are equidistributed the expected number of passes is $1 + o(1)$. (An unconditional guard follows from the irrationality measure $\mu(\pi) \le 7.11$ of Zeilberger–Zudilin [18], but only with $G = \Theta(N)$, which would sacrifice quasi-linearity; every implementation of binary BBP makes the same tradeoff.)

3 Complexity

3.1 Precomputation and per-term costs

$X = 5^n$ is computed by repeated squaring in $O(\mathsf{M}(N))$ bit operations (the $O(\log N)$ squarings act on geometrically growing operands, so the cost telescopes) and stored as $\approx 0.2902N$ bytes. The $O(N)$ modular exponentiations (Phases 1–2) cost $O(\log N)$ multiplications of $O(\log N)$-bit words each: $O(N \log N)$ word operations, i.e. $O(N \log^2 N \log\log N)$ bit operations; window reads are $O(1)$ words each.

3.2 Batched prefix residues

The one nontrivial cost is the family of prefix residues $\widetilde{Y}_t = (X \bmod 2^{J_t}) \bmod \mu_t$ over the $T = \Theta(N)$ band terms. Computed naively (a Horner pass over the stored string per term) each costs $O(J_t/w)$ word operations, a quadratic total if done term by term. It is however batchable with remainder-tree technology [12, 13]:

Theorem 7 (prefix-residue batching)**.** *Given the binary expansion of X (of length L) and pairs $(J_1, \mu_1), \dots, (J_T, \mu_T)$ with word-size moduli and the J_t in arithmetic progression, all residues $(X \bmod 2^{J_t}) \bmod \mu_t$ can be computed in $O\big(\mathsf{M}(L + Tw)\log T\big)$ bit operations.*

Proof sketch. Let δ be the common difference of the J_t (for the four families of (3), $\delta = 4$; run one instance per family) and let $c_t \in [0, 2^\delta)$ be the bits of X in $[J_t, J_{t+1})$. The prefix values $P_t := X \bmod 2^{J_t}$ obey the linear recurrence $P_{t+1} = P_t + 2^{J_t} c_t$, i.e. the pair $v_t = (P_t,\, 2^{J_t})^{\mathsf{T}}$ transforms as $v_{t+1} = A_t\, v_t$ with the word-size upper-triangular matrix

$$A_t = \begin{pmatrix} 1 & c_t \\ 0 & 2^\delta \end{pmatrix}, \qquad \text{so} \qquad \widetilde{Y}_t = \big[(A_{t-1} \cdots A_0)\, v_0\big]_1 \bmod \mu_t.$$

Computing a running product of word-size matrices reduced modulo a different word-size modulus at each leaf is precisely the *accumulating remainder tree* of Costa–Gerbicz–Harvey and Harvey–Sutherland [14, 15]: one product/remainder tree over the T leaves in which the partial products (which grow to $\Theta(L)$ bits at the root) are pushed down and reduced modulo the subtree modulus products (total size $\Theta(Tw)$). Each of the $O(\log T)$ levels costs $O(\mathsf{M}(L + Tw))$ by superadditivity of M, giving the bound. Naively the workspace is $O(L + Tw) = \Theta(N \log N)$ bits, held level by level; Proposition 8 reduces it to $\Theta(N)$. □

Proposition 8 ($\Theta(N)$ space via a remainder forest)**.** *The residues of Theorem 7, and likewise the V-phase merge of Section 4.2, can be computed in $\Theta(N)$ bits of working space while preserving the $O(N \log^3 N)$ time bound, by processing the T leaves as a* remainder forest*: partition them into*

$\Theta(\log N)$ *consecutive blocks of* $\Theta(T/\log N)$ *leaves each. Each block's modulus product occupies* $\Theta(N)$ *bits, and the prefix* $X \bmod 2^{J_t}$ *that every leaf in a block requires is a window of the read-only string* X*; the blocks are therefore independent and are processed sequentially, one resident at a time. Peak working space is thus* $O(N)$ *(one block's trees) on top of the read-only* X*, and the time is* $\Theta(\log N)$ *blocks* $\times\, O(\mathsf{M}(N)\log N) = O(N\log^3 N)$*, unchanged up to a constant. This is the space-reduction device of Harvey–Sutherland (accumulating remainder* forests*) [15].*

With $L = \Theta(N)$, $T = \Theta(N)$, $w = O(\log N)$ this is $O(\mathsf{M}(N\log N)\log N) \subseteq O(N\log^3 N)$, and Theorem 1 follows by summing the phase costs: the accumulating tree dominates, the modular exponentiations contribute $O(N\log^2 N\log\log N)$ bits, and everything else is smaller. In the word RAM the totals are $O(N\log^2 N)$ word operations for the tree and $O(N\log N)$ for the exponentiations, and Section 4 reduces the latter, and every other non-tree cost, to $O(N\log\log N)$.

3.3 Comparison

Method	Time	Space	Base	Status
BBP 1997 [1]	$\widetilde{O}(N)$	$O(\log N)$	2^r only	implemented
Plouffe 1996 [3]	$O(N^3\log^3 N)$	$O(\log N)$	any	implemented
Bellard 1997 [4]	$O(N^2)$	$O(\log N)$	any	implemented
Gourdon 2003 [5], Thm. 1	$O(N^2\log\log N/\log^2 N)$ [a]	$O(\log^2 N)$	10	implemented
Gourdon 2003 [5], Thm. 2	$O\big(\frac{N^2\log^3 N\log\log N}{m\log^2(N/m)}\big)$	$O(m)$	10	asserted only
Plouffe 2022 [6]	(no analysis; $N \le 10^8$)	tables B_{2n}	10, 2	implemented
This paper	$O(N\log^3 N)$	$\Theta(N)$ bits (shared, r/o)	any	implemented[b]

[a] word operations. [b] measured $N^{1.077}$ over $N \in [10^4, 10^7]$; Section 7.

4 Faster phases: sieve-batching and a 2-adic normal form

4.1 Sieve-batched exponentiation

The exponentiation workload of Sections 2–3 consists, per term, of powers of 2 whose exponents are *linear in the index* k (namely $2^{\pm 4k}$-type factors and 2^{-J} with J linear in k) and powers with *fixed* exponents (5^n and $2^{n+O(1)}$ modulo μ). Square-and-multiply prices each at $O(\log N)$ word multiplications: $O(N\log N)$ in total. Both kinds amortize.

Lemma 9 (Euler starters)**.** *Let* q *be an odd prime and* k_0 *minimal with* $q \mid 8k_0+1$*, and write* $8k_0+1 = qs$*; then* $s \le 7$ *is odd and*

$$2^{4k_0} \;\equiv\; \left(\tfrac{2}{q}\right)^{s} 2^{(s-1)/2} \pmod q,$$

computable in $O(1)$ *word operations. Analogous* $O(1)$ *formulas hold for the progressions* $8k+5$*,* $4k+3$*,* $2k+1$ *(using* $2^{qs-3} \equiv 2^{s-3}$ *etc.).*

Proof. $k_0 < q$ gives $s \le 7$; qs odd gives s odd. Then $4k_0 = (qs-1)/2 = s\cdot\frac{q-1}{2} + \frac{s-1}{2}$, and $2^{(q-1)/2} \equiv (\frac{2}{q}) \pmod q$ is Euler's criterion; the Legendre symbol is read off $q \bmod 8$. □

Lemma 10 (Fermat chain constants)**.** *For an odd prime* q*,* $2^{4q} \equiv 2^4 = 16 \pmod q$*, and likewise* $2^{-4q} \equiv 16^{-1} \pmod q$*.*

Theorem 11 (sieve-batched exponentiation)**.** *All 2-power residues with k-linear exponents, over all terms of* (3) *with $k \le K$, are computable in $O(K \log\log K)$ word operations; all fixed-exponent residues ($5^n \bmod \mu$, $2^{n+O(1)} \bmod \mu$) likewise in $O(K \log\log K)$ word operations, the per-term cost of the once-per-prime-power computations vanishing as $O(1/\log K)$. Hence the entire exponentiation phase costs $O(N \log\log N)$ word operations.*

Proof sketch. A segmented sieve over each progression delivers the factorization of every μ at $O(K \log\log K)$ total cost. Fix a prime power $P = q^e$ dividing some moduli; the indices k with $P \mid \mu(k)$ form an arithmetic progression mod P, along which $2^{4k} \bmod P$ is a geometric sequence with ratio $2^{4P} \bmod P$: by Lemmas 9–10 the starter and (for $e = 1$) the ratio are $O(1)$, and each further element costs one multiplication. The number of (term, prime-power) incidences is $\sum_{k \le K} \Omega'(\mu(k)) = O(K \log\log K)$ by Mertens; prime powers with $e \ge 2$ have $O(\sum_q K/q^2) = O(K)$ incidences and afford honest exponentiations for their rare starters. Fixed-exponent residues are computed once per *distinct* prime power (there are $O(K/\log K)$ of them, at $O(\log N)$ each), and every term's residue is assembled from its $O(\log\log K)$ cached prime-power components by CRT at $O(1)$ multiplications per incidence. □

Remark 12 (application to binary BBP). The classical BBP loop computes $16^{d-k} \bmod (8k + j)$ per term by square-and-multiply, and has since 1997. The structure required by Theorem 11 (fixed base, exponent linear in k, modulus on an arithmetic progression in k) is exactly the structure of that loop, so the same sieve-batching applies to hexadecimal/binary digit extraction of π and to every BBP-type computation. We are not aware of a prior appearance of this amortization. Our validation run (Section 7) measures 2.7× fewer multiplications already at $K = 2 \cdot 10^4$; the ratio scales as $\Theta(\log K/\log\log K)$.

4.2 The Bézout split and the 2-adic normal form

Lemma 13 (Bézout split)**.** *Let μ be odd, $J \ge 1$, $u = 2^{-J} \bmod \mu$, and $w = \mu^{-1} \bmod 2^J$. Then, exactly,*

$$\left\{\frac{X}{\mu 2^J}\right\} = \left\{\frac{(X \bmod \mu)\, u \bmod \mu}{\mu} + \frac{Xw \bmod 2^J}{2^J}\right\}.$$

Proof. With $v = (1 - u2^J)/\mu \in \mathbb{Z}$ we have $u2^J + v\mu = 1$, hence $X/(\mu 2^J) = Xu/\mu + Xv/2^J$; reduce each part mod 1 and note $v \equiv \mu^{-1} \pmod{2^J}$. □

The first summand is word arithmetic, and its ingredients $X \bmod \mu$ and $u = 2^{-J} \bmod \mu$ are exactly what Theorem 11 delivers at amortized $O(\log\log N)$: *the odd halves of all band terms join the sieved word phase.* The dyadic halves collapse:

Lemma 14 (dyadic consolidation)**.** *Let $J_{\max} = \max_t J_t$ and*

$$V \;=\; \sum_t \pm\, 2^{\,J_{\max} - J_t} \left(\mu_t^{-1} \bmod 2^{\,J_{\max}}\right) \;\bmod\; 2^{\,J_{\max}}.$$

Then $\sum_t \pm\{X(\mu_t^{-1} \bmod 2^{J_t})/2^{J_t}\} \equiv \{XV/2^{\,J_{\max}}\} \pmod 1$.

Proof. Write $\mu^{-1} \bmod 2^{J_{\max}} = (\mu^{-1} \bmod 2^J) + 2^J t$; then $X(\mu^{-1} \bmod 2^{J_{\max}})/2^J = X(\mu^{-1} \bmod 2^J)/2^J + Xt$ and the integer Xt vanishes mod 1. Summing and factoring $2^{-J_{\max}}$ gives the claim; replacing V by $V \bmod 2^{J_{\max}}$ changes the value by an integer multiple of X. □

The refined algorithm

The extractor now runs in four phases. The sieve phase (Theorem 11) produces the factorizations, chains, and caches in $O(N \log\log N)$ word operations. The word phase accumulates all easy terms and the odd halves of the band terms in fixed point, again in $O(N \log\log N)$ word operations. The V phase builds V by a balanced merge of numerator/denominator-product pairs modulo powers of two, using truncated multiplication at each node and one Newton–Hensel inversion mod $2^{J_{\max}}$ at the end; it costs $O(\mathsf{M}(N \log N) \log N)$ and is division-free. (An equivalent variant instead builds the exact sum $X \cdot (A/B)$, with $A/B = \sum_t \pm 1/(\mu_t 2^{J_t})$, by binary splitting, at the same asymptotic cost.) The final phase forms one product $XV \bmod 2^{J_{\max}}$, reads one window, and outputs the digit, in $O(\mathsf{M}(N))$. Every logarithm in Theorem 1 lives in the V phase.

5 Generalizations

Theorem 15. *Let ξ admit a BBP-type formula in base 2^r, $\xi = \sum_{k\ge 0} 2^{-rk}\, p(k)/q(k)$ with integer polynomials and q having word-size values, in the sense of [1]. Then the N-th decimal digit of ξ is computable in the time and space of Theorem 1.*

Proof. Termwise, $10^n 2^{-rk} p(k)/q(k) = 2^{n-rk} 5^n p(k)/q(k)$; folding the 2-part of $q(k)$ into the exponent yields the normal form $\pm 2^E 5^n p(k)/\mu$ with μ odd, and Lemmas 3–4 apply with numerator $p(k)X$ (h and u scale by the word-size $p(k)$ before the fixed-point division). All counts are as before. □

For example $\ln 2 = \sum_{k\ge 1} 1/(k2^k)$ gives, with $k = 2^s\mu$, terms $2^{n-k-s} 5^n/\mu$: the first quasi-linear-time decimal digit extractor for $\ln 2$. The same holds for π^2, $\ln^2 2$, Catalan-type combinations, and the rest of the base-2^r BBP zoo of [8].

Corollary 16 (any base)**.** *For any base $B \ge 2$, the N-th base-B digit of any base-2^r BBP constant is computable in $\widetilde{O}(N)$ time with $\Theta(N \log B)$ bits of read-only auxiliary space, namely the expansion of Q^{N-1} where Q is the odd part of B.*

Proof. $B^n = 2^{an} Q^n$ with Q odd; the normal form becomes $\pm 2^E Q^n/\mu$ and the argument is unchanged with $X = Q^n$. (For odd B, $a = 0$ and every non-easy term is band or tail.) □

In particular the base-5 digits of π are computable in quasi-linear time with a shared string $X = 5^{N-1}$: in that case *every* term with $k \ge 1$ is a band or tail term. This answers the question left open in [7] affirmatively at $\Theta(N)$-bit space: the terms on which his identity (3) fails (denominator $5^{2n+1-d+k}m$) are exactly our band terms, and Lemma 3 computes them. Whether polylogarithmic space suffices remains open, and our reliance on stored middle bits of 5^n suggests a connection to the apparent hardness of computing isolated middle bits of integer powers.

We also note that the framework accepts any Machin-type formula whose arctangent arguments are 2-5-smooth, such as the 1844 Strassnitzky–Dase identity $\pi/4 = \arctan\frac{1}{2} + \arctan\frac{1}{5} + \arctan\frac{1}{8}$ (equivalently $(2+i)(5+i)(8+i) = 65(1+i)$ in $\mathbb{Z}[i]$); by Størmer-type finiteness these smooth identities are scarce, and we use this one only as an independent correctness check.

6 The barrier

After Section 4, every phase except the construction of V runs in $O(N \log\log N)$ word operations; the V phase remains at the product-tree bound. This section characterizes the obstruction.

6.1 Status of lower bounds

No unconditional lower bound is available at any level: proving even $\Omega(N)$ for the digit itself would require ruling out closed forms for the digits of π, far beyond current techniques, an obstacle shared with binary BBP. Within the framework, $\Omega(N)$ is forced structurally: $\Theta(N)$ terms each contribute at $O(1)$ magnitude modulo 1, and the u-windows jointly tile $\Theta(N)$ bits of X. The natural conjectural floor is $N \log N$: every known route to the string of $X = 5^{N-1}$ costs $\Theta(\mathsf{M}(N))$, and $\mathsf{M}(N) = \Theta(N \log N)$ is believed optimal: the upper bound is Harvey–van der Hoeven [16], and a matching lower bound is known only conditionally, on a network-coding conjecture [17]; no reduction from multiplication to fixed-base powering is known, so even that conditional bound does not formally transfer. Sublinear time would require abandoning the series paradigm altogether, since every term matters at full weight modulo 1, and three decades of work on binary BBP have not produced $o(N)$ extraction in any base. Everything between the proven $O(N \log^3 N)$ and the conjectural $N \log N$ floor is the subject of what follows.

6.2 Equivalent formulations

The denominator product entering the V phase is, per family, a rising-factorial-type product: e.g. $\prod_{k\le K}(8k+1) = 8^{K+1}(1/8)^{\overline{K+1}}$. Computing such products (or V, which every known merge scheme routes through them) modulo $2^{\Theta(N)}$ below the product-tree bound $\mathsf{M}(n\log n)\log n$ is an instance of the classical factorial barrier. Borwein's factorization shaves the *product* to $\mathsf{M}(n\log n)\log\log n$ (Theorem 19); but for a single term of a general holonomic sequence, which is what V is, no method below the product tree is known, despite long attention [19, 20, 21].

A second formulation: $(\mathbb{Z}/2^m\mathbb{Z})^* = \langle -1\rangle \times \langle 3\rangle$, so $\prod_{k\le K}(1+8k) \equiv \pm 3^E \pmod{2^m}$ where

$$E = \sum_{k\le K} \operatorname{dlog}_3(1+8k) \mod 2^{m-2}, \qquad m = \Theta(N).$$

Computing the *single integer* E is thus a distillation of the multiplicative side of the barrier, and it falls (Theorem 19 below). Expanding $\operatorname{dlog}_3(1+8k) = \log_2(1+8k)/\log_2 3$ through the 2-adic logarithm turns E into $\sum_j c_j S_j(K)$ with power sums $S_j(K) = \sum_{k\le K} k^j$; Faulhaber's formula then prices the exchange in Bernoulli numbers, and the coefficient mass of every arrangement we tried is $\Theta(N^2)$ bits. A third formulation is the merge depth itself: summing T inhomogeneous word-size rationals below $\mathsf{M}(\text{total})\log T$.

A fourth formulation is analytic, and rests on the observation that Euler–Maclaurin summation, divergent-asymptotic over $\mathbb{R}$, is *exactly convergent* over $\mathbb{Z}_2$ for our summand.

Lemma 17 (2-adic Euler–Maclaurin)**.** *Let $f(x) = \operatorname{Log}_2(1+8x)$ (the 2-adic logarithm) and $F(x) = \big((1+8x)(f(x)-1)+1\big)/8$. Then, as an identity in $\mathbb{Z}_2$ with the right side absolutely convergent (the i-th correction has 2-adic valuation $\ge 6i-8$),*

$$\sum_{k=1}^{K} f(k) \;=\; F(K) + \tfrac{1}{2}f(K) + \sum_{i\ge 1} \frac{B_{2i}}{(2i)!}\Big(f^{(2i-1)}(K) - f^{(2i-1)}(0)\Big),$$

where $f^{(r)}(x) = (-1)^{r-1}(r-1)!\,8^r(1+8x)^{-r}$. *(Machine-validated exactly at* $(K,m) = (100,160)$ *and* $(257,224)$ 2*-adic bits.)*

Since the correction series is, up to elementary factors, the Stirling tail $\sum_i B_{2i}\,w^{2i-1}/\big(2i(2i-1)\big)$ evaluated at $w = 8/(1+8K)$ (valuation 3), the lemma says that E (hence $\prod_{k\le K}(1+8k) \bmod 2^m$, hence the barrier) *is* a single-point evaluation of Diamond's 2-adic log-gamma function [26] in its domain of convergent asymptotics, plus elementary terms; and conversely, such evaluations at arguments $8/(1+8K)$, $K = \Theta(m)$, reduce back to E-type sums by reading the identity in reverse. Choosing $K = 2^s$ and running the dyadic doubling recursion instead reproduces partial 2-adic Hurwitz zeta values at positive integers, confirming the same equivalence from the Iwasawa side (cf. [27]). Every known evaluation strategy for these p-adic special functions pays the tapered-Bernoulli mass $\Theta(N^2)$ bits or falls back to the product tree, but the reduction imports the problem into computational Iwasawa theory, where evaluation of p-adic L-functions is an active algorithmic topic, and progress there now transfers here.

Two further formulations matter below. Via the Coleman/Iwasawa formalism, E is a truncation of the Mazur measure's Γ-transform; computing it fast is the same problem as computing the Kubota–Leopoldt 2-adic zeta function to high precision, itself open in computational Iwasawa theory. And most elementary: since $\mathrm{Log}_2\, u = \lim_r (u^{2^r}-1)/2^r$, one has the machine-validated congruence

$$\sum_{k\le K}(1+8k)^{2^r} \;\equiv\; K \;+\; 2^r \sum_{k\le K} \mathrm{Log}_2(1+8k) \pmod{2^{\,2r+5}},$$

so the entire barrier is equivalent to a statement requiring no p-adic language at all: *compute* $\sum_{k\le K}(1+8k)^{2^r} \bmod 2^{2r}$*, with* $K, r = \Theta(N)$*, in* $\widetilde{O}(N)$ *time.*

Remark 18 (the analytic root, and why the exits are closed). The recurring Bernoulli mass has a classical cause: the generating function $x/(e^x-1)$ has infinitely many poles ($2\pi i k$, the poles of ζ), hence is *not holonomic*, so the Bernoulli sequence satisfies no polynomial recurrence, and every fast point-evaluation technology for series (binary splitting of P-recurrences, bit-burst, including its 2-adic version, which achieves quasi-linear precision for exactly the holonomic class [23]) requires such a recurrence; moreover no bit-burst analogue for the full differentially algebraic class can exist, since every polynomial-time-computable real is a value of a polynomial-ODE solution [24] and the time hierarchy theorem forbids universal quasi-linear evaluation. Over $\mathbb{R}$ this is harmless because integral representations route around the coefficients; over $\mathbb{Z}_2$ the quadrature rigidity above closes that road. The wall is thus the confluence of two facts: *no recurrence, and no integral end-run.* Nor can the formula be shopped away: a series whose denominators were purely geometric would sum to a rational number, so transcendence forces the polynomial factor in every BBP-type denominator, which forces moduli sweeping arithmetic progressions, which forces the formulations above. Two further resources are accounted for: quantum period-finding offers nothing, since the per-element task (one 2-adic discrete logarithm) is already classically easy and exact summation to $2^{-\Theta(N)}$ defeats amplitude-estimation; and the mass is parallelizable in low depth (iterated multiplication and division modulo 2^m lie in DLOGTIME-uniform TC^0 [25]), so at scale it is a cost in work, not in wall-clock time.

6.3 A crack: the multiplicative formulations fall

Theorem 19 (sub-product-tree evaluation of the multiplicative formulations)**.** $\prod_{k\le K}(8k+1) \bmod 2^m$ *(and hence* E*, the power sums* $\sum_{k\le K}(1+8k)^{2^r} \bmod 2^{2r}$*, and the log-gamma values of Lemma 17) are computable in* $O(\mathsf{M}(N)\log N) = O(N\log^2 N)$ *bit operations.*

Proof sketch. Borwein's factorial method [19] transfers. The exponent of an odd prime p in the product is $e_p = \sum_j \#\{k \le K : p^j \mid 8k+1\}$, a Legendre-type count on an arithmetic progression, $O(1)$ per prime power after a sieve of cost $O(N \log\log N)$. The distinct-prime mass is $\sum_{p \le 8K} \lg p = \Theta(N)$ bits: the factored representation compresses the $\Theta(N \log N)$-bit term list to $\Theta(N)$ bits, with multiplicities riding as exponents. Assemble $\prod p^{e_p} \bmod 2^m$ by exponent-bit scheduling: per bit level, one truncated product tree over the participating primes (level masses decay geometrically from $\Theta(N)$; total $O(\mathsf{M}(N) \log N)$) and one squaring mod 2^m ($O(\log N)$ squarings in all). Finally $E = \mathrm{Log}_2(\prod)/\mathrm{Log}_2 9$, the 2-adic logarithm being quasi-linear in precision [23]. Machine-validated at six sizes through $K = 3 \cdot 10^6$: exact agreement with the direct product, and the measured time ratio grows monotonically in the factored method's favor. □

Three consequences. First, these formulations are *not* constructively equivalent: they split into a multiplicative cluster (products, E, power sums, and log-gamma/truncated Kubota–Leopoldt values, now all $O(N \log^2 N)$) and the *additive core* V itself, the consolidated sum $\sum_t \pm 2^{\,J_{\max}-J_t} \mu_t^{-1}$, whose numerator admits no prime factorization. The log-gamma evaluation is, to our knowledge, the first single-prime *sub-product-tree* evaluation of Diamond's 2-adic log-gamma at the arguments $8/(1+8K)$. (Morita's Γ_p is implemented in PARI/GP, Magma and SageMath, PARI/GP's `lngamma` computes Diamond's function for $|x|_2 > 1$, and Γ_p is computable in average polynomial time over all $p \le X$ [35, 36]; new here is the sub-product-tree cost at a single prime, specialized to these arguments.) Second, Theorem 1 is unchanged, since the digit pipeline consumes V: the barrier now stands, sharper, on the additive core alone. (A complete inventory of the approaches we examined against it, some fifteen routes across real-analytic, 2-adic, mod-p, and circuit-theoretic settings, each with its terminal obstruction, together with the leads we consider live, is available from the author.)

6.4 The additive core as an isolated harmonic number

Proposition 20 (the additive core is an isolated harmonic number)**.** *Let* $\Pi(t) = \prod_{k \le K}(8k+1+t)$. *Stripping the dyadic weights* $2^{\,J_{\max}-J_t}$ *from* V *leaves the numerator* $\Pi'(0) = \sum_k \prod_{j \ne k}(8j+1)$, *and*

$$\frac{\Pi'(0)}{\Pi(0)} = \sum_{k \le K} \frac{1}{8k+1} \;\equiv\; \tfrac{1}{8}\Big(\psi_2\big(K+1+\tfrac{1}{8}\big) - \psi_2\big(\tfrac{1}{8}\big)\Big) \pmod{2^m},$$

a 2-adic digamma value: equivalently the x^1*-coefficient of a product of* $\Theta(N)$ *linear forms, and, via* $n!\,H_n = |s(n+1,2)|$, *an unsigned Stirling number of the first kind. Thus, up to the weights, the additive core is an isolated harmonic number modulo* 2^m. *(Verified exactly in the accompanying code.)*

The product $\Pi(0)$ is *smooth*, so Borwein's factorization applies (Theorem 19); its logarithmic derivative $\Pi'(0)$ is a generic integer carrying a large prime factor (already 961 bits at $K = 200$): *differentiation destroys the factorization the crack runs on*, which is precisely why the multiplicative formulations fall while the additive core does not. This problem is well studied and open. For an isolated harmonic number modulo a prime power, binary splitting [28] is the best known method; the factorial's one-logarithm shave is not known to transfer to the sum (posed as open in [29]); and the single-prime cost of $(p-1)! \bmod p^2$ is $\widetilde{O}(\sqrt{p})$ [14], beating which is open and tied to the absence of polynomial-time point counting [30]. The one machine that carries *both* a product and its sum of partial products (a p-adic Γ value and, as we observe,

its logarithmic derivative) is the accumulating remainder tree of Costa–Kedlaya–Roe [35, 36], but only amortized over all primes $p \le X$; whether that amortization can be traded for single-argument precision at the fixed prime 2 is Open Problem 21.

The core is also, structurally, an incomplete exponential sum: as a partial sum of inverses $\sum_{k\le K}(8k+1)^{-1} \bmod 2^m$ it sits on the *wild* side of the tame/wild divide. Quadratic exponential sums are polynomial-time (Gauss sums), whereas specific degree-≥ 3 families are #P-hard even at a fixed prime power [31]; single Kloosterman sums have no known efficient classical evaluation [32]; and partial Kloosterman sums converge in law to random processes, even to high prime-power moduli [33, 34]. This is evidence rather than proof. It places the additive core among problems for which no fast algorithm is known, and suggests that completeness, rather than the field-versus-ring distinction, is what makes the complete multiplicative sums tame.

6.5 The barrier is not cryptographically protected

Fast factorials modulo an *arbitrary* integer m would factor m (Shamir's classical argument [22], in the unit-cost arithmetic model: binary-search $\gcd(k! \bmod m, m)$), which is often cited as evidence that factorials must be hard. That shield does not cover our instance: the modulus is the fixed prime power $2^{\Theta(N)}$, where the gcd argument is vacuous. To our knowledge nothing but the absence of an idea protects $\prod(8k+1) \bmod 2^{\Theta(N)}$, E, or V. We consider this the most attackable formulation and state it as the target:

Open Problem 21. Compute the additive core $V = \sum_t \pm 2^{\,J_{\max}-J_t}\big(\mu_t^{-1} \bmod 2^{J_{\max}}\big) \bmod 2^{J_{\max}}$ (equivalently its numerator $\sum_t \pm 2^{\,J_{\max}-J_t}\prod_{s\ne t}\mu_s \bmod 2^{J_{\max}}$), in $O(\mathsf{M}(N)\log N)$ bit operations. Via the pipeline of Section 4 this immediately yields decimal (and binary) digit extraction of π in $O(N\log^2 N)$. The denominator product is computable within that budget (Theorem 19); the *sum* is the open half, and no factoring-hardness or information-theoretic argument protects it.

Conjecture 22 (mass conservation, additive form)**.** Every algorithm computing the additive core V of Open Problem 21 by ring operations over $\mathbb{Z}/2^{O(N)}$ whose intermediate values are coefficient arrays in the bases occurring in this section (moments, Bernoulli/Iwasawa coefficients, inverse-power sums, symmetric functions, or per-prime residue systems) reads or writes $\Omega(N^2/\log^{O(1)} N)$ bits in total. (The original form of this conjecture also covered E; Theorem 19 resolved that case exactly as the conjecture's guidance predicted, by an unlisted representation.)

The routes of the table, spanning real-analytic, 2-adic, mod-p and circuit-theoretic bases, are the conjecture's evidence; the compactness observation above shows it cannot be proven information-theoretically, and an unconditional proof would amount to an explicit superlinear circuit lower bound, beyond current techniques. Its practical content is guidance: a resolution of Open Problem 21 must operate outside every basis the conjecture names.

Remark 23 (structural exclusions)*.* Beyond the basis-by-basis evidence, four broad strategies for Open Problem 21 can be excluded outright.

The first is formula shopping. For every BBP family with denominators $dt+1$ ($d = 4, \dots, 16$) the truncation-defect of the associated Dwork-type ratio obeys the same law, $\min_j v_2 = s + v_2(d) - 1$ at truncation depth 2^s, so changing the formula shifts a constant but never the slope, and no base-2^r formula makes the additive core easier. We verified this law from the Frobenius side, as the valuation of the classical Dwork residual $f_{s+1}(x)f_{s-1}(x^2) - f_s(x)f_s(x^2)$ for $f = {}_2F_1(\frac{1}{8}, 1; \frac{9}{8}; x)$; it means Dwork/Frobenius descent supplies only $\Theta(\log N)$ 2-adic digits at the

depth $s = \Theta(\log N)$ that N terms require, and the one amplifier, an excellent Frobenius lift to modulus p^{2s}, requires p odd [37] and so is vacuous at $p = 2$.

The second is depth-doubling by AGM- or Landen-type iteration. The duplication law of the underlying Lerch-type sum, $\Phi(z, 1, a) = \frac{1}{2}\Phi(z^2, 1, \frac{a}{2}) + \frac{z}{2}\Phi(z^2, 1, \frac{a+1}{2})$, is exact and truncation-compatible but branches in its parameter: $\log N$ doublings hold N parameters, reproducing the product tree. The classical logarithm admits an AGM because its transformation fixes a single parameter, whereas the truncated object branches.

The third is incomplete reciprocity. The real-analytic incomplete Landsberg–Schaar/theta reciprocity carries an unavoidable transcendental (Fresnel) remainder [9]; a 2-adic analogue would need an algebraic remainder in $\mathbb{Z}/2^m$, and a noise-floor-calibrated lattice-reduction search (linear and quadratic-character ζ_8-duals, two parameter points, planted-relation controls recovered at height $O(1)$) finds the candidate space empty below the random-lattice threshold.

The fourth is algebraic cross-scale relations in general. The same calibrated search over truncated logarithms at five dyadic depths, their pairwise products, and their exponentials finds no integer relation between height $O(1)$ (where the known functional equation is recovered exactly) and the noise floor. Any resolution of Open Problem 21 must therefore be non-algebraic in the truncation scale, consistent with Conjecture 22, whose failure, if it fails, will not be by a scale-recursion.

7 Implementation and verification

The algorithm (the pipeline of Sections 2–4, with the division-free construction of V realized by symbolic shifts and per-node precision caps) is implemented in a few hundred lines of Python over GMP integers (`decimal_bbp_extract.py`, covering both π and $\ln 2$), accompanied by an independent C++/OpenMP implementation of the band identity used for cross-checking (`three_phase_extract.cpp`) and by standalone exact validators for every identity in the paper (`validation/`). The components cross-validate: the batched band construction reproduces a direct per-term accumulator bit-for-bit across the verification battery, and the independent code paths agree on every digit window reported below.

Correctness

All of the following were verified against digits computed independently by integer Chudnovsky binary splitting (and, for $\ln 2$, by binary splitting of $2\,\mathrm{artanh}(1/3)$): positions 1–60 of π continuously (8-digit windows); π at $N = 10^2, 10^3, 5 \cdot 10^3, 10^4, 10^5, 10^6$; $\ln 2$ at 1–40 and $N = 10^2, 10^3, 10^4, 10^5$; agreement at every tested position between the BBP-16 path and the independent Strassnitzky–Dase path; and an exact-rational audit of 397 random band/tail terms (worst fixed-point deviation $2^{-153.6}$, bound 2^{-142}). At $N = 10^6$ the extractor returns 130927562832 at positions $10^6..10^6{+}11$, matching the reference. At $N = 10^7$ it returns 725915133612 at positions $10^7..10^7{+}11$; we verified this window against two independently published digit corpora: Google's 100-trillion-digit computation (served at `pi.delivery`, with the index convention confirmed by probing the leading digits) and the 200-million-digit corpus at `angio.net`, whose substring search locates the window at exactly position 10^7 after the decimal point.

Identity-level validations

The mechanisms of Sections 4–6 are backed by exact machine checks: the chain/CRT mechanism of Theorem 11 reproduces $2^{4k} \bmod (8k+1)$ for all $k \le 2 \cdot 10^4$ (with the Euler-criterion consequence $2^{4k} \equiv 1$ for prime $8k+1$ confirmed throughout, and $2.7\times$ fewer word multiplications than per-term square-and-multiply at that size); Lemma 13 was verified exactly on 300 random band terms; Lemma 14 was verified exactly against the termwise dyadic sum; Lemma 17 was verified exactly at $(K, m) = (100, 160)$ and $(257, 224)$; the Mazur-measure (Γ-transform) form and the extraction $E = \mathrm{Log}(\prod u)/\log 9$ were verified exactly at 96 bits; and the power-sum congruence was verified exactly at $(K, r) = (200, 120)$ (all 245 bits), including recovery of the Log-sum from the elementary power sum alone.

Measurements

These timings are from a Windows 11 desktop, our implementation single-threaded on CPython 3.10 over GMP. As external baseline we compiled a straightforward implementation of Gourdon's method [5], the strongest published decimal extractor, faithful to the published algorithm but not micro-optimized, and raced it on the same machine; its digits agree with our verified windows throughout. The constant-factor gap below is therefore only indicative; the claim rests on the *asymptotic* separation, which follows from the analyses (N^2/polylog for Gourdon versus $N^{1+o(1)}$ here) and is independent of implementation quality.

N	Gourdon [5]	this paper	auxiliary string
10^4	0.59 s	0.06 s	2.9 KB
10^5	32.3 s	0.77 s	29 KB
10^6	2069 s	8.9 s	290 KB
10^7	—	111.6 s	2.9 MB

The fitted exponents are $\approx N^{1.8}$ for Gourdon's method (quadratic-class, as its analysis predicts) against $N^{1.077}$ for the present algorithm over three decades with no upward bend: the quasi-linear regime of Theorem 1 is visible directly, in interpreted code, and the gap over the strongest published method, already $232\times$ at $N = 10^6$, grows without bound. At $N = 10^7$ the phase split is 64 s for the exponentiation loop, 47 s for the entire consolidated band (the terms on which every previous decimal method was quadratic) and 0.05 s to build X. The sieve-batched exponentiation of Theorem 11 is validated at the same scale in operation count (identical digits at $N = 10^7$, with multiplication counts as predicted); realizing it in wall clock belongs to compiled implementations, where cost tracks multiplications rather than interpreter statements.

8 Open problems

(1) Open Problem 21: the additive core, the one remaining logarithm. (2) Decimal digit extraction in sublinear space, the original problem of [1], remains open; is there a lower-bound obstruction linking it to the complexity of isolated middle bits of 5^n? (3) The per-term independence invites a distributed record attempt: verifying isolated *decimal* positions of record computations of π (a practical need noted by record holders) requires exactly this primitive. (4) Zudilin's Gaussian-integer series can presumably be treated by the same band identity over $\mathbb{Z}[i]$

with a shared string for $(1-2i)^d$; carrying this out would give a second, structurally different quasi-linear base-5 extractor.

Acknowledgments

The author thanks the maintainers of the computational records and surveys cited here.

Use of AI assistance

Computational experiments, the literature survey, and manuscript preparation were carried out with assistance from an AI system (Anthropic Claude). All mathematical identities were independently machine-validated by the exact-arithmetic scripts in the accompanying repository, and each externally cited result was checked against its primary source. The author is solely responsible for the results and their correctness.

References

[1] D. Bailey, P. Borwein, S. Plouffe, *On the rapid computation of various polylogarithmic constants*, Math. Comp. **66** (1997), 903–913.

[2] J. M. Borwein, D. Borwein, W. F. Galway, *Finding and excluding b-ary Machin-type individual digit formulae*, Canad. J. Math. **56** (2004), 897–925.

[3] S. Plouffe, *On the computation of the n'th decimal digit of various transcendental numbers*, 1996; arXiv:0912.0303.

[4] F. Bellard, *Computation of the n'th digit of pi in any base in $O(n^2)$*, 1997, `https://bellard.org/pi/pi_n2/pi_n2.html`.

[5] X. Gourdon, *Computation of the n-th decimal digit of π with low memory*, unpublished manuscript, February 11, 2003, `http://numbers.computation.free.fr/Constants/Algorithms/nthdecimaldigit.pdf`.

[6] S. Plouffe, *A formula for the n'th decimal digit or binary of π and powers of π*, arXiv:2201.12601 (2022).

[7] W. Zudilin, *A BBP-style computation for π in base 5*, arXiv:2409.10097 (2024). (First arXiv version titled "... in base 10".)

[8] D. H. Bailey, *A compendium of BBP-type formulas for mathematical constants*, manuscript, updated 2023, `https://www.davidhbailey.com/dhbpapers/bbp-formulas.pdf`.

[9] A. Fedotov, F. Klopp, *An exact renormalization formula for Gaussian exponential sums and applications*, arXiv:0909.3079 (2009).

[10] R. P. Brent, *Fast multiple-precision evaluation of elementary functions*, J. ACM **23** (1976), 242–251.

[11] E. Salamin, *Computation of π using arithmetic-geometric mean*, Math. Comp. **30** (1976), 565–570.

[12] A. Borodin, R. Moenck, *Fast modular transforms*, J. Comput. System Sci. **8** (1974), 366–386.

[13] D. J. Bernstein, *Scaled remainder trees*, manuscript, 2004-08-20, `https://cr.yp.to/arith/scaledmod-20040820.pdf`.

[14] E. Costa, R. Gerbicz, D. Harvey, *A search for Wilson primes*, Math. Comp. **83** (2014), 3071–3091.

[15] D. Harvey, A. V. Sutherland, *Computing Hasse–Witt matrices of hyperelliptic curves in average polynomial time*, LMS J. Comput. Math. **17** (2014), 257–273.

[16] D. Harvey, J. van der Hoeven, *Integer multiplication in time $O(n \log n)$*, Ann. of Math. **193** (2021), 563–617.

[17] P. Afshani, C. B. Freksen, L. Kamma, K. G. Larsen, *Lower bounds for multiplication via network coding*, in: ICALP 2019, LIPIcs **132**, 10:1–10:12.

[18] D. Zeilberger, W. Zudilin, *The irrationality measure of π is at most* 7.103205334137..., Mosc. J. Comb. Number Theory **9** (2020), no. 4, 407–419; arXiv:1912.06345.

[19] P. B. Borwein, *On the complexity of calculating factorials*, J. Algorithms **6** (1985), 376–380.

[20] V. Strassen, *Einige Resultate über Berechnungskomplexität*, Jahresber. Deutsch. Math.-Verein. **78** (1976/77), 1–8.

[21] A. Bostan, P. Gaudry, É. Schost, *Linear recurrences with polynomial coefficients and application to integer factorization and Cartier–Manin operator*, SIAM J. Comput. **36** (2007), 1777–1806.

[22] A. Shamir, *Factoring numbers in $O(\log n)$ arithmetic steps*, Inform. Process. Lett. **8** (1979), 28–31.

[23] X. Caruso, M. Mezzarobba, N. Takayama, T. Vaccon, *Fast evaluation of some p-adic transcendental functions*, in: ISSAC 2021; arXiv:2106.09315.

[24] O. Bournez, D. S. Graça, A. Pouly, *Polynomial time corresponds to solutions of polynomial ordinary differential equations of polynomial length*, J. ACM **64** (2017), no. 6, art. 38.

[25] W. Hesse, E. Allender, D. A. Mix Barrington, *Uniform constant-depth threshold circuits for division and iterated multiplication*, J. Comput. System Sci. **65** (2002), 695–716.

[26] J. Diamond, *The p-adic log gamma function and p-adic Euler constants*, Trans. Amer. Math. Soc. **233** (1977), 321–337.

[27] L. C. Washington, *Introduction to Cyclotomic Fields*, 2nd ed., GTM 83, Springer, 1997.

[28] R. P. Brent, P. Zimmermann, *Modern Computer Arithmetic*, Cambridge Univ. Press, 2010.

[29] F. Johansson, *How (not) to compute harmonic numbers*, blog post, 2009, `https://fredrikj.net/blog/2009/02/how-not-to-compute-harmonic-numbers/`.

[30] D. Harvey, *Counting points on hyperelliptic curves in average polynomial time*, Ann. of Math. **179** (2014), 783–803.

[31] J.-Y. Cai, X. Chen, R. J. Lipton, P. Lu, *On tractable exponential sums*, in: FAW 2010, LNCS **6213** (2010), 148–159.

[32] P. Bruin, *On quantum computation of Kloosterman sums*, arXiv:1807.03600 (2018).

[33] E. Kowalski, W. F. Sawin, *Kloosterman paths and the shape of exponential sums*, Compos. Math. **152** (2016), 1489–1516.

[34] D. Milićević, S. Zhang, *Distribution of Kloosterman paths to high prime power moduli*, Trans. Amer. Math. Soc. Ser. B **10** (2023), 636–669.

[35] E. Costa, K. S. Kedlaya, D. Roe, *Hypergeometric L-functions in average polynomial time*, in: ANTS-XIV, Open Book Ser. **4** (2020), 143–159.

[36] E. Costa, K. S. Kedlaya, D. Roe, *Hypergeometric L-functions in average polynomial time, II*, Res. Number Theory **11** (2025), art. 32; arXiv:2310.06971.

[37] F. Beukers, M. Vlasenko, *Dwork crystals III: from excellent Frobenius lifts towards super-congruences*, Int. Math. Res. Not. IMRN (2023), 20433–20483.